

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2**



Stack Dan Queue

Oleh:

Vivaldi Fachmy Fahlevi

NIM. 2510817210017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	1
DAFTAR ISI	2
DAFTAR GAMBAR.....	1
DAFTAR TABEL	2
SOAL 1	1
A. Source Code.....	1
B. Pembahasan	4
SOAL 2.....	6
A. Source Code.....	8
B. Output Program.....	10
C. Pembahasan	11
SOAL 3.....	13
A. Source Code.....	13
B. Pembahasan	16
SOAL 4.....	19
A. Source Code.....	21
B. Output Program.....	23
C. Pembahasan	24
TAUTAN GIT	26

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Soal 2 File prob2.cpp	10
Gambar 2 Screenshot Output Program Soal 4 File prob2.cpp	23

DAFTAR TABEL

Tabel 1 Source Code File stack.h	1
Tabel 2 Source Code File stack.cpp	2
Tabel 3 Source Code File prob1.cpp	8
Tabel 4 Source Code File queue.h.....	13
Tabel 5 Source Code File queue.cpp	14
Tabel 6 Source Code File prob2.cpp	21

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code File stack.h

1	<code>#ifndef STACK_H</code>
2	<code>#define STACK_H</code>
3	
4	<code>#define MAX 100</code>
5	
6	<code>struct Stack {</code>
7	<code> int data[MAX];</code>
8	<code> int* top;</code>
9	<code>};</code>
10	
11	<code>void init(Stack* s);</code>
12	<code>bool isEmpty(const Stack* s);</code>
13	<code>bool isFull(const Stack* s);</code>
14	<code>void push(Stack* s, int value);</code>
15	<code>void pop(Stack* s);</code>
16	<code>int peek(const Stack* s);</code>
17	
18	<code>#endif</code>

Tabel 2 Source Code File stack.cpp

```
1  #include "stack.h"
2  #include <stdexcept>
3
4  void init(Stack* s) {
5      s->top = s->data - 1;
6  }
7
8  bool isEmpty(const Stack* s) {
9      return s->top < s->data;
10 }
11
12 bool isFull(const Stack* s) {
13     return s->top == s->data + MAX - 1;
14 }
15
16 void push(Stack* s, int value) {
17     if (isFull(s)) {
18         throw std::overflow_error("Stack penuh");
19     }
20     s->top++;
21     *(s->top) = value;
22 }
23
24 void pop(Stack* s) {
25     if (isEmpty(s)) {
26         throw std::underflow_error("Stack kosong");
27     }
28     s->top--;
29 }
30
31 int peek(const Stack* s) {
32     if (isEmpty(s)) {
33         throw std::underflow_error("Stack kosong");
34     }
35     return *(s->top);
```

36	}
----	---

B. Pembahasan

File stack.h

Pada file **stack.h**, didefinisikan struktur dasar dari stack beserta deklarasi fungsi-fungsi yang akan digunakan. Pada baris [4], terdapat konstanta **MAX** yang menentukan kapasitas maksimum stack yaitu 100 elemen. Hal ini berarti stack memiliki ukuran tetap (statis) dan tidak dapat menampung lebih dari jumlah tersebut.

Selanjutnya pada baris [6–9], dideklarasikan sebuah `struct Stack` yang berfungsi sebagai wadah penyimpanan data. Pada baris [7], `data[MAX]` merupakan array yang digunakan untuk menyimpan elemen stack. Kemudian pada baris [8], terdapat pointer `top` yang berfungsi untuk menunjuk posisi elemen teratas dalam stack. Penggunaan pointer di sini menunjukkan bahwa pendekatan yang digunakan adalah pointer-based, dimana posisi elemen ditentukan berdasarkan alamat memori, bukan indeks secara langsung.

Pada baris [11–16], dideklarasikan fungsi-fungsi utama yang akan diimplementasikan pada file `stack.cpp`. Fungsi `init` baris [11] digunakan untuk menginisialisasi stack agar berada pada kondisi awal (kosong). Fungsi `isEmpty` baris [12] digunakan untuk memeriksa apakah stack kosong, sedangkan `isFull` baris [13] untuk memeriksa apakah stack sudah penuh.

Fungsi `push` baris [14] berfungsi untuk menambahkan elemen ke dalam stack, sedangkan `pop` baris [15] digunakan untuk menghapus elemen teratas. Terakhir, fungsi `peek` baris [16] digunakan untuk melihat nilai elemen teratas tanpa menghapusnya. Dengan adanya deklarasi ini, file `stack.h` berperan sebagai interface atau penghubung antara program utama dengan implementasi stack yang terdapat pada `stack.cpp`.

File stack.cpp

Pada file `stack.cpp`, seluruh fungsi yang telah dideklarasikan diimplementasikan secara lengkap. Pada baris [4–6], fungsi `init` digunakan untuk menginisialisasi stack dengan cara mengatur pointer `top` ke posisi sebelum elemen pertama, yaitu `s->data - 1`. Hal ini menandakan bahwa stack dalam kondisi kosong karena belum ada elemen yang disimpan.

Pada baris [8–10], fungsi `isEmpty` memeriksa apakah stack kosong dengan membandingkan posisi `top` terhadap alamat awal array `data`. Jika `top` berada sebelum `data`, maka stack dianggap kosong. Logika ini benar karena pada kondisi awal `top` memang diset ke `data - 1`.

Pada baris [12–14], fungsi `isFull` digunakan untuk mengecek apakah stack sudah penuh. Kondisi penuh terjadi ketika pointer `top` berada pada elemen terakhir array, yaitu `data + MAX - 1`. Hal ini memastikan bahwa tidak ada elemen tambahan yang dapat dimasukkan lagi ke dalam stack.

Pada baris [16–22], fungsi push digunakan untuk menambahkan elemen ke dalam stack. Pertama, pada baris [17–19], dilakukan pengecekan apakah stack sudah penuh menggunakan `isFull`. Jika penuh, maka program akan melempar `exception overflow_error`. Jika tidak penuh maka pada baris [20], pointer `top` digeser ke depan (menaik), dan pada baris [21], nilai baru disimpan pada posisi yang ditunjuk oleh `top`. Algoritma ini benar karena elemen selalu ditambahkan di posisi teratas.

Pada baris [24–29], fungsi pop digunakan untuk menghapus elemen dari stack. Pada baris [25–27], dilakukan pengecekan apakah stack kosong menggunakan `isEmpty`. Jika kosong, maka akan dilempar `exception underflow_error`. Jika tidak kosong, maka pada baris [28], pointer `top` digeser ke belakang (menurun), yang berarti elemen teratas dihapus. Pendekatan ini benar karena tidak perlu menghapus data secara fisik, cukup dengan mengubah posisi `top`.

Pada baris [31–36], fungsi peek digunakan untuk melihat elemen teratas tanpa menghapusnya. Sama seperti pop, pada baris [32–34] dilakukan pengecekan kondisi kosong. Jika stack tidak kosong, maka pada baris [35], nilai yang ditunjuk oleh pointer `top` dikembalikan. Hal ini sesuai dengan konsep stack, dimana elemen teratas dapat diakses langsung.

Dari segi kebenaran algoritma, implementasi ini telah memenuhi prinsip **LIFO (Last In First Out)**, dimana elemen terakhir yang dimasukkan melalui fungsi `push` akan menjadi elemen pertama yang diambil melalui fungsi `pop`. Hal ini dapat dilihat dari penggunaan pointer `top` yang selalu bergerak maju saat penambahan data dan mundur saat penghapusan data.

Dari segi kompleksitas, seluruh operasi pada stack ini memiliki kompleksitas waktu **$O(1)$** karena setiap fungsi hanya melakukan operasi sederhana tanpa perulangan. Kompleksitas ruang yang digunakan adalah **$O(n)$** , dimana n adalah kapasitas maksimum stack (`MAX`), karena menggunakan array statis sebagai tempat penyimpanan data

SOAL 2

SI PALUI DAN MESIN HITUNG ANEH

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- N Adalah jumlah simbol dalam ekspresi
- A_i Adalah nilai bilangan pada input
- Simbol yang valid hanya :
Bilangan bulat
Operator : + , - , * , /

Format Input

n A1 A2
... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

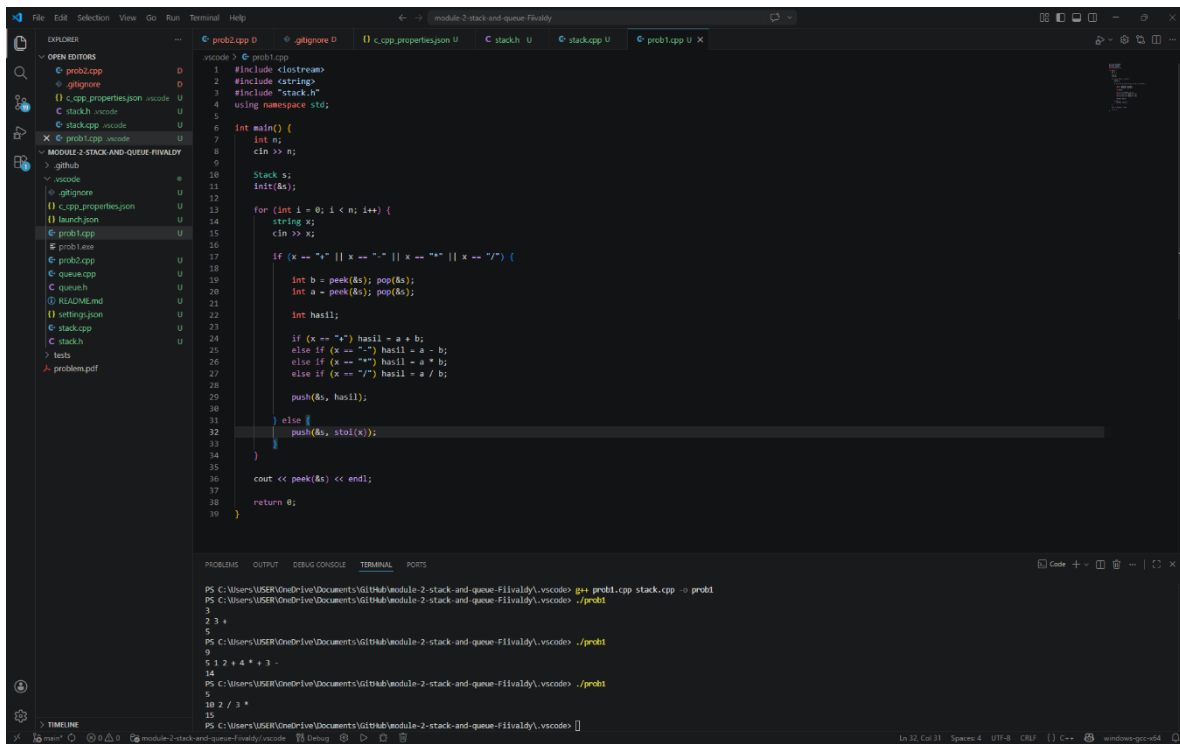
A. Source Code

Tabel 3 Source Code File probl.cpp

```
1  #include <iostream>
2  #include <string>
3  #include "stack.h"
4  using namespace std;
5
6  int main() {
7      int n;
8      cin >> n;
9
10     Stack s;
11     init(&s);
12
13     for (int i = 0; i < n; i++) {
14         string x;
15         cin >> x;
16
17         if (x == "+" || x == "-" || x == "*" || x ==
18             "/" ) {
19             int b = peek(&s); pop(&s);
20             int a = peek(&s); pop(&s);
21
22             int hasil;
23
24             if (x == "+") hasil = a + b;
25             else if (x == "-") hasil = a - b;
26             else if (x == "*") hasil = a * b;
27             else if (x == "/") hasil = a / b;
28
29             push(&s, hasil);
30
31         } else {
32             push(&s, stoi(x));
```

```
33         }  
34     }  
35  
36     cout << peek(&s) << endl;  
37  
38     return 0;  
39 }
```

B. Output Program



```
1 #include <iostream>
2 #include <string>
3 #include "stack.h"
4 using namespace std;
5
6 int main() {
7     int n;
8     cin >> n;
9
10    Stack s;
11    Init(&s);
12
13    for (int i = 0; i < n; i++) {
14        string x;
15        cin >> x;
16
17        if (x == "+" || x == "-" || x == "*" || x == "/" ) {
18            int b = peek(&s); pop(&s);
19            int a = peek(&s); pop(&s);
20
21            int hasil;
22
23            if (x == "+") hasil = a + b;
24            else if (x == "-") hasil = a - b;
25            else if (x == "*") hasil = a * b;
26            else if (x == "/") hasil = a / b;
27
28            push(&s, hasil);
29        }
30        else {
31            push(&s, stoi(x));
32        }
33    }
34
35    cout << peek(&s) << endl;
36    return 0;
37 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Filivaldy\vscode> g++ prob1.cpp stack.cpp -o prob1
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Filivaldy\vscode> ./prob1
3
2 3 +
5
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Filivaldy\vscode> ./prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Filivaldy\vscode> ./prob1
5
10 2 / 3 *
19
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Filivaldy\vscode>
```

Gambar 1 Screenshot Output Program Soal 2 File prob2.cpp

C. Pembahasan

Program pada Soal 2 sangat bergantung pada fungsi-fungsi yang telah diimplementasikan pada `stack.h` dan `stack.cpp`. Pada saat program memanggil `init(&s)` di baris [11], fungsi ini mengatur pointer `top` ke posisi awal (sebelum elemen pertama), sehingga `stack` siap digunakan dalam kondisi kosong.

Pada saat membaca bilangan, fungsi `push(&s, value)` yang dipanggil pada baris [28] dan [31] akan menambahkan elemen ke dalam `stack` dengan cara menggeser pointer `top` ke depan dan menyimpan nilai baru pada posisi tersebut. Sebaliknya, pada saat operator ditemukan, fungsi `peek(&s)` dan `pop(&s)` pada baris [19–20] digunakan untuk mengambil dan menghapus elemen teratas. Fungsi `peek` mengembalikan nilai tanpa menghapusnya, sedangkan `pop` hanya menggeser pointer `top` ke belakang untuk menghapus elemen secara logika.

Fungsi `isEmpty` dan `isFull` yang terdapat dalam `stack.cpp` secara tidak langsung juga berperan penting karena digunakan di dalam `push` dan `pop` untuk mencegah kondisi tidak valid. Jika terjadi kondisi seperti `stack` kosong saat `pop` atau penuh saat `push`, maka program akan melempar exception menggunakan `throw`, sehingga program menjadi lebih aman dan tidak mengalami error yang tidak terkontrol.

Dengan demikian, implementasi `stack` pada Soal 1 terbukti berhasil digunakan sebagai dasar penyelesaian Soal 2. Seluruh operasi berjalan dengan kompleksitas waktu $O(1)$ untuk setiap operasi `stack`, sehingga keseluruhan program tetap efisien. Hal ini menunjukkan bahwa struktur data `stack` sangat cocok digunakan untuk menyelesaikan permasalahan evaluasi ekspresi postfix.

Program pada Soal 2 berfungsi untuk menghitung operasi matematika dengan cara membaca urutan angka dan operator dari kiri ke kanan, kemudian memprosesnya menggunakan struktur data `stack`. Pada baris [7–8], program membaca jumlah simbol `n` yang akan diproses. Kemudian pada baris [10–11], dibuat sebuah variabel `Stack s` dan dilakukan inisialisasi menggunakan fungsi `init(&s)` agar `stack` berada dalam kondisi kosong sebelum digunakan.

Selanjutnya pada baris [13–30], dilakukan perulangan sebanyak `n` kali untuk membaca setiap simbol yang dimasukkan oleh pengguna. Pada baris [14–15], setiap simbol dibaca dalam bentuk string `x`, karena input dapat berupa angka maupun operator. Pada baris [17], program memeriksa apakah simbol tersebut merupakan operator (+, -, *, atau /).

Jika simbol adalah operator, maka pada baris [19–20], program mengambil dua nilai teratas dari `stack` menggunakan fungsi `peek` dan `pop`. Nilai pertama yang diambil disimpan ke dalam variabel `b`, dan nilai berikutnya ke dalam `a`. Urutan ini penting karena operasi matematika harus dilakukan dalam bentuk `a operator b`.

Pada baris [22–26], dilakukan proses perhitungan sesuai operator yang dibaca. Hasil perhitungan tersebut kemudian disimpan kembali ke dalam `stack` menggunakan fungsi `push`

pada baris [28]. Hal ini sesuai dengan cara kerja mesin hitung yang dijelaskan pada soal, dimana hasil sementara disimpan kembali untuk digunakan pada operasi berikutnya.

Jika simbol bukan operator, maka pada baris [30–32], simbol tersebut dianggap sebagai bilangan dan dikonversi dari string ke integer menggunakan fungsi `stoi`, kemudian dimasukkan ke dalam stack menggunakan `push`.

Setelah seluruh simbol selesai diproses, pada baris [35], program menampilkan hasil akhir dengan mengambil elemen teratas dari stack menggunakan fungsi `peek`. Nilai ini merupakan hasil akhir dari seluruh operasi karena dalam metode postfix hanya akan tersisa satu nilai di akhir proses.

Secara keseluruhan, dalam algoritma ini setiap operator selalu bekerja pada dua operand terakhir yang dimasukkan, sehingga sesuai dengan prinsip evaluasi postfix. Kompleksitas waktu dari algoritma ini adalah $O(n)$ karena setiap simbol diproses satu kali, sedangkan kompleksitas ruang adalah $O(n)$ karena stack dapat menyimpan hingga n bagian.

SOAL 3

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 4 Source Code File queue.h

1	<code>#ifndef QUEUE_H</code>
2	<code>#define QUEUE_H</code>
3	
4	<code>#define MAX 100</code>
5	
6	<code>struct Queue {</code>
7	<code> int data[MAX];</code>
8	<code> int* front;</code>
9	<code> int* rear;</code>
10	<code>};</code>
11	
12	<code>void init(Queue* q);</code>
13	<code>bool isEmpty(const Queue* q);</code>
14	<code>bool isFull(const Queue* q);</code>
15	<code>void enqueue(Queue* q, int value);</code>
16	<code>void dequeue(Queue* q);</code>
17	<code>int front(const Queue* q);</code>
18	<code>int back(const Queue* q);</code>
19	
20	<code>#endif</code>

Tabel 5 Source Code File queue.cpp

```
1  #include "queue.h"
2  #include <stdexcept>
3
4  void init(Queue* q) {
5      q->front = q->data;
6      q->rear = q->data - 1;
7  }
8
9  bool isEmpty(const Queue* q) {
10     return q->rear < q->front;
11 }
12
13 bool isFull(const Queue* q) {
14     return q->rear == q->data + MAX - 1;
15 }
16
17 void enqueue(Queue* q, int value) {
18     if (isFull(q)) {
19         throw std::overflow_error("Queue penuh");
20     }
21     q->rear++;
22     *(q->rear) = value;
23 }
24
25 void dequeue(Queue* q) {
26     if (isEmpty(q)) {
27         throw std::underflow_error("Queue kosong");
28     }
29     q->front++;
30 }
31
32 int front(const Queue* q) {
33     if (isEmpty(q)) {
```

```
34         throw std::underflow_error("Queue kosong");
35     }
36     return *(q->front);
37 }
38
39 int back(const Queue* q) {
40     if (isEmpty(q)) {
41         throw std::underflow_error("Queue kosong");
42     }
43     return *(q->rear);
44 }
```

B. Pembahasan

File queue.h

Pada file queue.h, didefinisikan struktur dasar dari queue beserta deklarasi fungsi-fungsi yang akan digunakan. Pada baris [4], terdapat konstanta MAX yang menentukan kapasitas maksimum queue yaitu sebanyak 100 elemen. Hal ini menunjukkan bahwa queue menggunakan array statis dengan ukuran tetap.

Pada baris [6–10], dideklarasikan sebuah struct Queue sebagai tempat penyimpanan data. Pada baris [7], data[MAX] merupakan array yang digunakan untuk menyimpan elemen-elemen queue. Selanjutnya pada baris [8] dan [9], terdapat pointer front dan rear yang masing-masing berfungsi untuk menunjuk elemen paling depan dan paling belakang dalam queue. Penggunaan pointer ini menunjukkan bahwa pendekatan yang digunakan adalah pointer-based, dimana posisi elemen ditentukan berdasarkan alamat memori, bukan indeks secara langsung.

Pada baris [12–18], dideklarasikan fungsi-fungsi utama yang akan diimplementasikan pada file queue.cpp. Fungsi init (baris 12) digunakan untuk menginisialisasi queue agar berada dalam kondisi kosong. Fungsi isEmpty baris [13] berfungsi untuk memeriksa apakah queue kosong, sedangkan isFull baris [14] digunakan untuk memeriksa apakah queue sudah penuh.

Fungsi enqueue baris [15] digunakan untuk menambahkan elemen ke bagian belakang queue, sedangkan dequeue baris [16] digunakan untuk menghapus elemen dari bagian depan. Kemudian fungsi front baris [17] digunakan untuk melihat elemen paling depan tanpa menghapusnya, dan back baris [18] digunakan untuk melihat elemen paling belakang. Dengan adanya deklarasi ini, file queue.h berfungsi sebagai antarmuka yang menghubungkan program utama dengan implementasi queue pada queue.cpp.

File queue.cpp

Pada file `queue.cpp`, seluruh fungsi yang telah dideklarasikan pada `queue.h` diimplementasikan secara lengkap. Pada baris [4–7], fungsi `init` digunakan untuk menginisialisasi queue. Pointer `front` diatur ke awal array (`q->data`), sedangkan pointer `rear` diatur ke posisi sebelum elemen pertama (`q->data - 1`). Hal ini menandakan bahwa queue masih kosong karena belum ada elemen yang masuk.

Pada baris [9–11], fungsi `isEmpty` digunakan untuk memeriksa apakah queue kosong dengan membandingkan posisi pointer `rear` dan `front`. Jika `rear` berada sebelum `front`, maka queue dianggap kosong. Logika ini benar karena saat awal inisialisasi memang `rear` berada sebelum `front`.

Pada baris [13–15], fungsi `isFull` digunakan untuk mengecek apakah queue sudah penuh. Kondisi penuh terjadi ketika pointer `rear` sudah mencapai elemen terakhir dalam array, yaitu `data + MAX - 1`. Hal ini memastikan bahwa tidak ada ruang lagi untuk menambahkan elemen baru.

Pada baris [17–23], fungsi `enqueue` digunakan untuk menambahkan elemen ke dalam queue. Pada baris [18–20], dilakukan pengecekan apakah queue sudah penuh menggunakan `isFull`. Jika penuh, maka program akan melempar exception `overflow_error`. Jika tidak penuh, maka pada baris [21], pointer `rear` digeser ke depan, dan pada baris [22], nilai baru disimpan pada posisi tersebut. Proses ini sesuai dengan konsep queue, dimana elemen baru selalu ditambahkan di bagian belakang.

Pada baris [25–30], fungsi `dequeue` digunakan untuk menghapus elemen dari bagian depan queue. Pada baris [26–28], dilakukan pengecekan apakah queue kosong menggunakan `isEmpty`. Jika kosong, maka program akan melempar exception `underflow_error`. Jika tidak kosong, maka pada baris [29], pointer `front` digeser ke depan, yang berarti elemen paling depan telah dihapus secara logika. Pendekatan ini efisien karena tidak perlu memindahkan elemen lain dalam array.

Pada baris [32–37], fungsi `front` digunakan untuk melihat elemen paling depan tanpa menghapusnya. Jika queue kosong, akan dilempar exception. Jika tidak, maka nilai pada alamat yang ditunjuk oleh pointer `front` dikembalikan. Sedangkan pada baris [39–44], fungsi

back digunakan untuk melihat elemen paling belakang dengan cara mengembalikan nilai yang ditunjuk oleh pointer rear.

Secara keseluruhan, implementasi ini mengikuti prinsip FIFO (First In First Out), dimana elemen pertama yang masuk akan menjadi elemen pertama yang keluar. Hal ini dibuktikan dengan operasi enqueue yang menambahkan elemen di belakang dan dequeue yang menghapus elemen dari depan. Semua operasi seperti enqueue, dequeue, front, dan back memiliki kompleksitas waktu $O(1)$ karena hanya melibatkan perubahan pointer tanpa perulangan. Kompleksitas ruang juga efisien yaitu $O(1)$ untuk setiap operasi, karena tidak ada alokasi memori tambahan selain array yang telah ditentukan di awal.

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k A_1

A_2 ...

A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

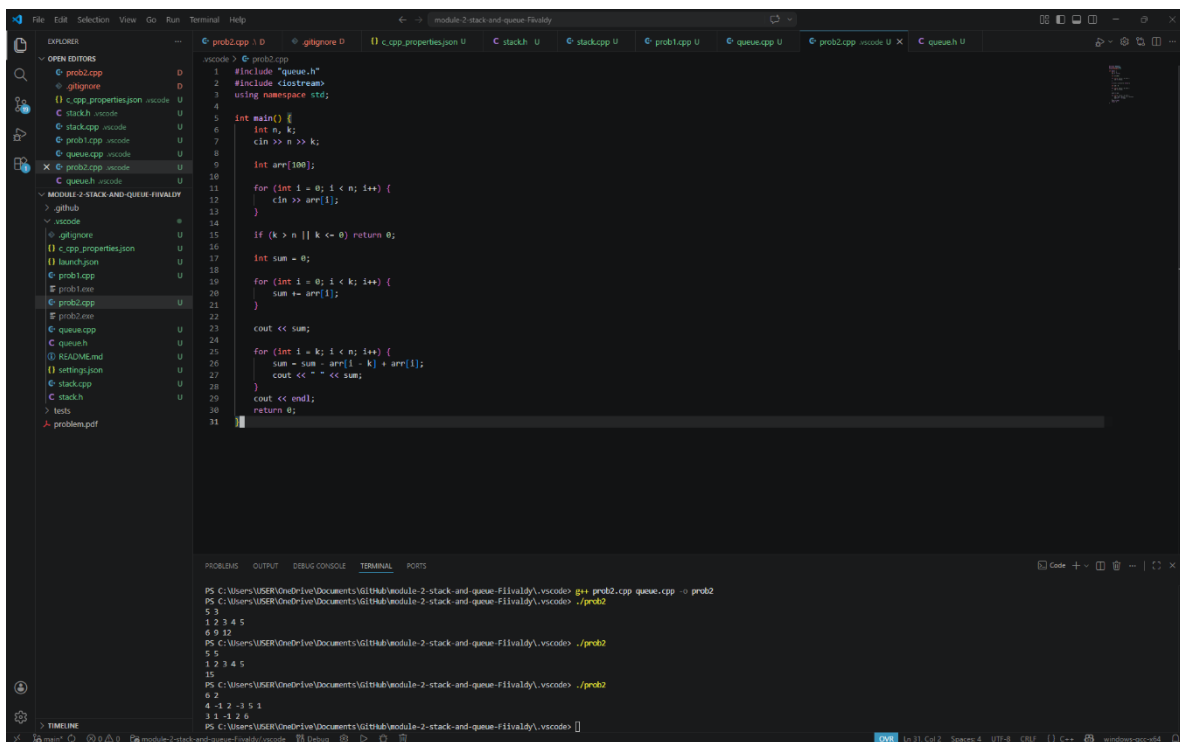
Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 6 Source Code File prob2.cpp

```
1  #include "queue.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      int n, k;
7      cin >> n >> k;
8
9      int arr[100];
10
11     for (int i = 0; i < n; i++) {
12         cin >> arr[i];
13     }
14
15     if (k > n || k <= 0) return 0;
16
17     int sum = 0;
18
19     for (int i = 0; i < k; i++) {
20         sum += arr[i];
21     }
22
23     cout << sum;
24
25     for (int i = k; i < n; i++) {
26         sum = sum - arr[i - k] + arr[i];
27         cout << " " << sum;
28     }
29     cout << endl;
30     return 0;
31 }
```


B. Output Program



```
1 #include "queue.h"
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int n, k;
7     cin >> n >> k;
8     int arr[100];
9
10    for (int i = 0; i < n; i++) {
11        cin >> arr[i];
12    }
13
14    if (k > n || k <= 0) return 0;
15
16    int sum = 0;
17
18    for (int i = 0; i < k; i++) {
19        sum += arr[i];
20    }
21
22    cout << sum;
23
24    for (int i = k; i < n; i++) {
25        sum = sum - arr[i - k] + arr[i];
26        cout << " " << sum;
27    }
28
29    cout << endl;
30    return 0;
31 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Finally\.vscode> g++ prob2.cpp queue.cpp -o prob2
5 3
1 2 3 4 5
6 9 12
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Finally\.vscode> ./prob2
5 5
1 2 3 4 5
15
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Finally\.vscode> ./prob2
6 7
4 -1 2 -3 5 1
3 1 -4 2 6
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-Finally\.vscode> |
```

Gambar 2 Screenshot Output Program Soal 4 File prob2.cpp

C. Pembahasan

Program pada file prob2.cpp digunakan untuk menghitung jumlah dari sejumlah elemen yang berurutan dalam sebuah array dengan menggunakan teknik *sliding window*. Pada baris [6–7], program membaca dua buah input yaitu n sebagai jumlah elemen dan k sebagai ukuran jendela (jumlah elemen yang dijumlahkan dalam satu kelompok).

Selanjutnya pada baris [9], dideklarasikan array `arr[100]` yang digunakan untuk menyimpan data angka yang dimasukkan oleh pengguna. Pada baris [11–13], dilakukan perulangan untuk membaca sebanyak n elemen dan menyimpannya ke dalam array tersebut.

Pada baris [15], terdapat pengecekan kondisi untuk memastikan nilai k valid. Jika k lebih besar dari n atau kurang dari atau sama dengan nol, maka program langsung berhenti dengan `return 0`. Hal ini penting untuk mencegah kesalahan dalam proses perhitungan.

Pada baris [17], variabel `sum` diinisialisasi dengan nilai 0. Variabel ini akan digunakan untuk menyimpan jumlah dari elemen-elemen dalam satu jendela. Kemudian pada baris [19–21], dilakukan perhitungan jumlah awal untuk k elemen pertama dalam array. Proses ini disebut sebagai *window pertama*.

Hasil dari perhitungan awal tersebut kemudian ditampilkan pada baris [23]. Setelah itu, pada baris [25–28], program mulai menerapkan teknik *sliding window*. Pada setiap iterasi, nilai `sum` diperbarui dengan cara mengurangi elemen yang keluar dari jendela (`arr[i - k]`) dan menambahkan elemen baru yang masuk ke dalam jendela (`arr[i]`). Dengan cara ini, program tidak perlu menghitung ulang seluruh elemen dalam setiap jendela, sehingga lebih efisien.

Setiap hasil perhitungan jendela ditampilkan secara berurutan dalam satu baris pada baris [27], dipisahkan dengan spasi. Terakhir, pada baris 29, program mencetak baris baru agar output lebih rapi, dan pada baris [30] program selesai dijalankan.

Secara keseluruhan, Dalam algoritma ini setiap langkah hanya menggeser jendela satu posisi ke kanan dan memperbarui jumlah berdasarkan perubahan elemen yang masuk dan keluar. Kompleksitas waktu dari algoritma ini adalah $O(n)$ karena setiap elemen hanya diproses satu kali, sedangkan kompleksitas ruang adalah $O(1)$ karena hanya menggunakan

variabel tambahan tanpa alokasi memori baru yang signifikan. Dengan demikian, metode ini lebih efisien dibandingkan cara biasa yang menghitung ulang setiap jendela dari awal.

TAUTAN GIT

<https://github.com/Fiivaldy/PRAKTIKUM-ALGORITMA-DAN-STRUKTUR-DATA.git>